

Upgrade 1 — Replace Loop-Bound Proxy With Process-Scoped Connection Pools

Problem

Current proxy creates clients per event loop.

Effects:

loop change → new client
new client → new pool
old pools remain alive

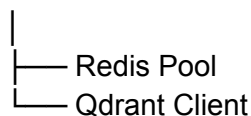
This produces **ghost connection pools** and uncontrolled socket growth.

Correct production model

Connections must be **process-scoped**, not loop-scoped.

Architecture:

Worker Process



All requests inside that worker reuse the same pool.

Implementation

Create infrastructure clients during FastAPI startup.

Example:

```
app.state.redis = redis.from_url(
    REDIS_URL,
    decode_responses=True,
    max_connections=50
)

app.state.qdrant = AsyncQdrantClient(
    url=QDRANT_URL
)
```

Access them via dependency:

```
def get_redis(request: Request):  
    return request.app.state.redis
```

Benefits:

- 1 pool per worker
- bounded connections
- no ghost pools
- predictable scaling

Upgrade 2 — Explicit Lifecycle Management

Problem

Your system currently relies on **process termination** to clean up resources.

Nothing closes:

- Redis pools
- Qdrant sessions
- SQLAlchemy engine
- threads

This causes resource leakage during:

- container restart
- worker reload
- rolling deployments

Production approach

Infrastructure must be **closed explicitly on shutdown**

Implementation

Add shutdown logic to lifespan.

Example:

```
async def lifespan(app: FastAPI):

    app.state.redis = redis.from_url(...)
    app.state.qdrant = AsyncQdrantClient(...)

    yield

    await app.state.redis.close()
    await app.state.qdrant.close()
    await engine.dispose()
```

Benefits:

- clean container shutdown
- no orphan sockets
- stable rolling deployments

Upgrade 3 — Define Infrastructure Connection Budgets

Problem

Your system currently has **no connection limits**.

Everything uses defaults:

- redis pool
- postgres pool
- qdrant http connections

This causes unpredictable failures when traffic grows.

Production rule

Define a **connection budget**.

Example deployment:

workers = 4
redis pool = 40
postgres pool = 30

Total Redis connections:

$$4 \times 40 = 160$$

Well below typical Redis limit:

maxclients $\approx$ 10000

Implementation

Redis:

```
redis.from_url(  
    REDIS_URL,  
    max_connections=40,  
    socket_timeout=5,  
    retry_on_timeout=True  
)
```

Postgres:

pool_size=30
max_overflow=10

Qdrant:

Limit concurrency via semaphore:

```
qdrant_limit = asyncio.Semaphore(50)
```

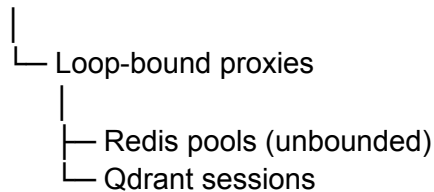
Benefits:

predictable scaling
no infrastructure exhaustion
stable latency

Resulting Architecture After Fixes

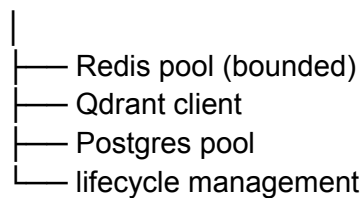
Current:

FastAPI



After upgrades:

Worker



System behavior becomes:

predictable

stable

production safe

The Reality

Your RAG pipeline itself is already well designed.

The missing pieces are **infrastructure discipline**:

pool ownership

resource lifecycle

connection budgeting

Once those three controls are added, the architecture becomes **production-grade for thousands of concurrent users**.